TEXT MINING

CA 2 Assignment

Report
LIAR2 Dataset

Submitted By: Abhinav Tyagi
20028460@mydbs.ie

Submitted To:

Prof. Kunwar Madan

Contents

Google Colab Link 3

Introduction 3

Data Preparation 3

1. Data Understanding 3
2. Concatenation/full_statement 4
3. Defining Real and Fake news/New_label 4
4. Data Cleaning and Processing 5
5. Limitations of the Approach 6

Model Creation and Evaluation 7

1. Classification models 7
2. Evaluation Metric 7
3. Models' evaluation and choice 8

Model Recommendation for Real World 11

1. Model Recommended 11
2. Improvements/ Alternatives to the approach applied 11
3. Improvement approach applied 12

Real-Time Prediction Using Saved Model 15

Google Colab Link

Click here to view Colab file :)

Introduction

Fake news refers to inaccurate or deceptive content that is presented in the guise of legitimate news. It typically manifests in several distinct forms. One category involves intentionally fabricated narratives designed to influence public opinion or alter significant societal outcomes, such as election results. Another common type includes content that reflects ideological bias or serves specific agendas such as propaganda, which distorts information to promote viewpoints. Additionally, some misinformation spreads unintentionally, often driven by the human tendency to share emotionally charged or sensational stories. Although this form may not stem from deliberate deceit, it can still propagate more rapidly than verified, factual reporting.

Detecting fake news is crucial to prevent misinformation, protect public opinion, and uphold trust in media and democratic processes.

Data Preparation

1. Data Understanding

- Imported necessary libraries: numpy, pandas, nltk which is used for text mining.
- Reading the dataset using read_csv imported from pandas
- Used **.info()** function to check for null values and data type, only one column '**state_info'** has missing values.
- Handling the missing data by filling with blank spaces.

```python
# importing libraries
import numpy as np # Numerical and Data Handling
import pandas as pd
import re , nltk #natural language tool kit library
# Downloading NLTK resources needed for text processing
nltk.download('stopwords')
nltk.download('punkt_tab')
nltk.download('omw-1.4')
nltk.download('wordnet')
# HTML and Lemmatization
from bs4 import BeautifulSoup
from nltk.stem import WordNetLemmatizer
from nltk.corpus import stopwords
from sklearn.feature_extraction.text import TfidfVectorizer  # Converts text to TF-IDF vectors
from sklearn import metrics # Model Evaluation
from sklearn.model_selection import StratifiedKFold
from sklearn.svm import LinearSVC
import joblib
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import  LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import recall_score , confusion_matrix
from sklearn.ensemble import  RandomForestClassifier
import seaborn as sns
import matplotlib.pyplot as plt
```

2. Concatenation/full_statement

- Concatenated only on the focused columns '**statement**' and '**justification**' to avoid overly complex input text.
- The **statement** contains the actual claim or news being made.
- The **justification** provides reasoning or context that was used during fact-checking.
- Together, they offer the **most direct and relevant textual evidence** for detecting whether the information is fake or real.
- Metadata Fields like speaker, context, state_info, speaker_decription add noise or bias.

```python
# Concatenating 'statement' and 'justification' for unified text analysis
data['full_statement'] =data['statement']+ " " +data['justification']
```

3. Defining Real and Fake news/New_label

- **Fake News (0):** Labels 0 (Pants on Fire), 1 (False), and 2 (Mostly False)
- **Real News (1):** Labels 3 (Half True), 4 (Mostly True), and 5 (True)

4. Data Cleaning and Processing

- Creating a function 'cleaner' for text cleaning.
  Key steps involved:
  (1) Removed HTML entities by using BeautifulSoup
  (2) Substituted '@, http://,https://,www' etc. and non-alphabetic characters with white spaces using re.sub function.
  (3) Converted document into token form using word_tokenize.
  (4) Used lambda function to filter out and remove stop words.
  (5) Used WordNetLemmatizer for lemmatizing tokens.
  (6) Apply cleaner function on full_statement and a new column '**cleaned_statement'** created in the form of tokens.
  (7) Used **join,** to join tokens into strings, as TF-IDF does not accept the input in token format.
  (8) Dividing the input (cleaned_statements) and target (New_label) column
  (9) Used TfidfVectorizer with min_dif = 0.0016 and ngram_range (1,3) which incluses unigram, bigram, & trigram, each ngram (unigram, bigram, & trigram) must be present in at least 30 documents for it to be considered as a feature (18240*0.0016 = 30)
  (10) Used .fit() to fit the input feature into TF IDF vectorizer and then transformed it and saved vocabulary into new_vocabulary.csv using .to_csv and used value_counts to check predicting class imbalance.

```python
# Cleaner function for Preprocessing the Text
def cleaner(fake_news):
  soup = BeautifulSoup(fake_news,'lxml') # Remove HTML entities
  souped =soup.get_text()
  re1 = re.sub(r"(@|https://|www|http://|\\x)\S*"," ",souped)  # Remove URLs, handles
  re2 = re.sub('[^a-zA-Z]+',' ',re1)  # Keep only alphabetic characters
  token = nltk.word_tokenize(re2) # Tokenize text into words
  lower_case = [t.lower() for t in token] # Lowercase all tokens
  stop_words = set(stopwords.words('english'))  # Get English stopwords
  filtered_result = list(filter(lambda l:l  not in stop_words,lower_case)) # Remove stopwords

  wordnet_lemmatizer = WordNetLemmatizer()  # Lemmatize (verb form)
  lemmas = [wordnet_lemmatizer.lemmatize(t,'v') for t in filtered_result]
  return lemmas
```

```python
# Apply cleaner function to all full statements
data['cleaned_statements'] = data.full_statement.apply(cleaner)
data = data[data['cleaned_statements'].map(len)>0] # Remove rows where cleaned statements are empty
print("Printing first 5 rows of dataframe showing original and cleaned statements") # Show cleaned vs. original text
print(data[['full_statement','cleaned_statements']].head())

# Dropping unneeded columns to simplify model input
data.drop(['id','full_statement','date','speaker','subject','speaker_description','state_info','true_counts','mostly_true_counts','half_true
data['cleaned_statement'] = [' '.join(row) for row in data['cleaned_statements'].values] # Joining tokens back into cleaned text strings

# Assigning final input and target variable
df= data['cleaned_statement']
Y = data['New_label']

#TF-IDF Vectorization
tfidf = TfidfVectorizer(min_df= 0.0016, ngram_range=(1,3))
tfidf.fit(df)
data_tfidf = tfidf.transform(df)

# Save vocabulary to a CSV
pd.DataFrame(pd.Series(tfidf.get_feature_names_out())).to_csv('news_vocalbulary.csv', header = False, index = False)

# printing shape of Tfidf matrix
print("Shape of TFIDF matrix shape : ",data_tfidf.shape)
```

5. Limitations of the Approach

- Removing stop words and punctuations may result in the loss of text information, which might be useful for detecting the fake news.
- **Potential information loss from dropping columns like speaker, context, or subject.**
- The TF-IDF and ngram does not provide details about the exact position of tokens. (syntactics)
- Dataset is slightly imbalanced.
- It is also not capable of understanding the sarcasm or nuanced text which is common in fake news.
- Manual thresholding in TF-IDF (min_df) may exclude rare but important n-grams.

Model Creation and Evaluation

1. Classification models

(1) **Linear SVC:** A strong linear baseline for text classification. Linear Support Vector Classifier (SVC) a linear model with Kernel = 'Linear' and C = 1.
(2) **Naïve Bayes Classifier:** A linear model, fast and effective for text, though it assumes feature independence.
(3) **Logistic Regression:** A linear model robust for high-dimensional data, good margin maximization.
(4) **Random Forest Classifier:** A non – linear Ensemble-based model for handling complex relationships.

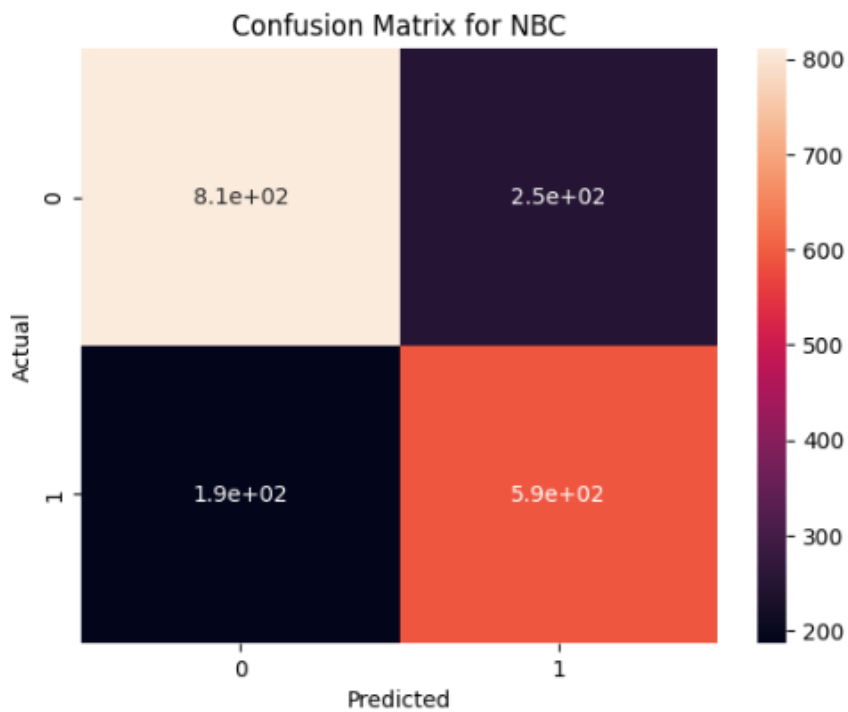
2. Evaluation Metric

- **Business Concern:** The social media company wants to detect false information/news.
- **Recall was prioritized to capture more fake news instances, reducing the risk of leaving misinformation undetected. 10-fold Stratified Cross-Validation was used.**
- Recall measures the ability of a model to correctly identify all relevant instances of a class. In our case, it's the percentage of actual fake news items that the model successfully flagged as fake.
- Accuracy cannot be used as an Evaluation Metric because the dataset is not balanced. As for precision, it is also important as it helps in detecting the FP (when the model predicts the Real news as Fake news. In fake news detection, a false negative (missing a fake news post) is riskier than a false positive (wrongly flagging a real post).
- High recall ensures that fewer fake news articles go undetected, making the system safer and more effective in a real-world social media setting.
- Ideal for a **flagging system**, where posts can still go to human moderators for further verification.

3. Models' evaluation and choice

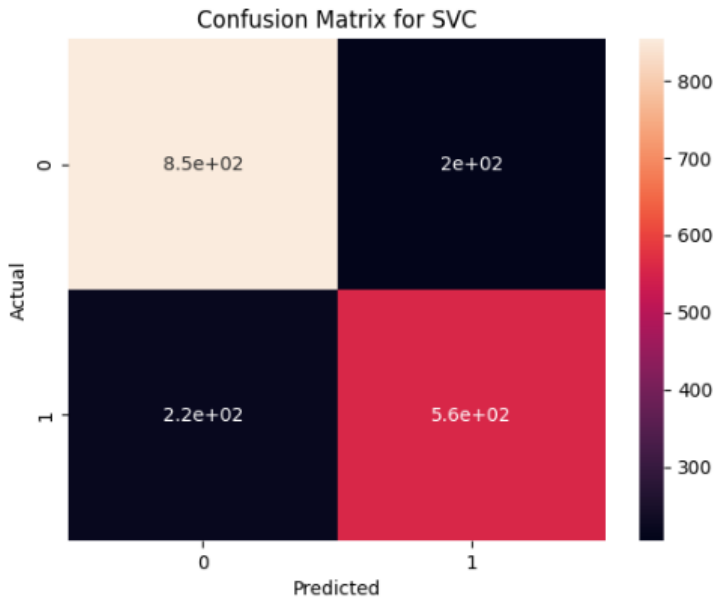
1) **Naïve Bayes (Recall: 0.7568)** Naïve Bayes achieved the highest recall, which suggests it is most effective at detecting Fake news (class 0) with **FN =187**.
   a) It works well with high-dimensional, sparse text data.
   b) It tends to be biased toward recall, especially when classes are slightly imbalanced (LIAR2 dataset is slightly imbalanced).
   c) However, it may overfit specific word patterns and doesn't handle word dependencies or context well.

```
[[811 248]
 [187 590]]
```



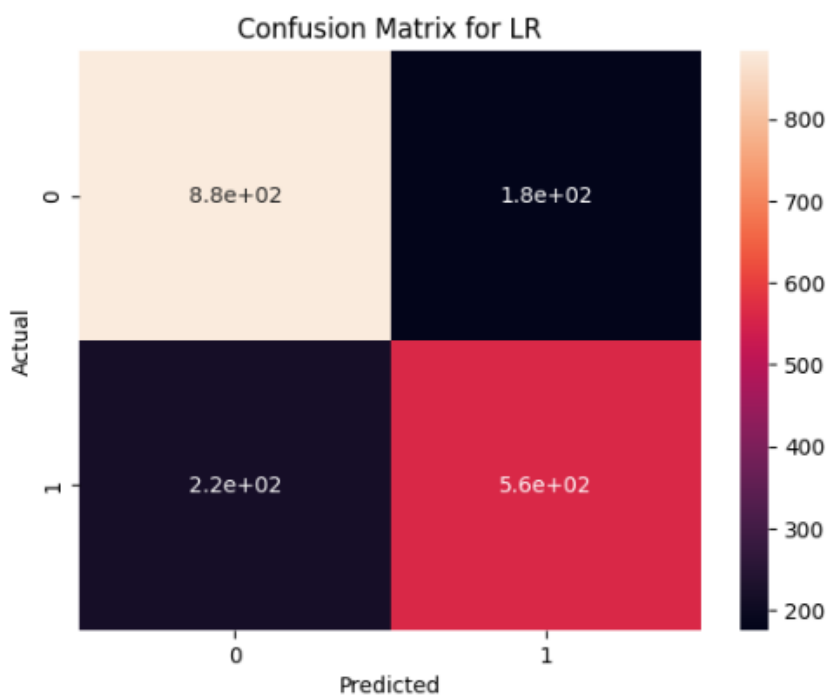
2) **SVC (Linear Kernel) (Recall: 0.7345)** SVC performed solidly, slightly behind Naive Bayes. It's generally robust for text classification with **FN =220**:
   a) Its ability to find optimal hyperplanes in high-dimensional space.
   b) Resistance to overfitting.
   c) It balances precision and recall better than Naive Bayes, though it's slightly slower to train.

```
[[854 205]
 [220 557]]
```



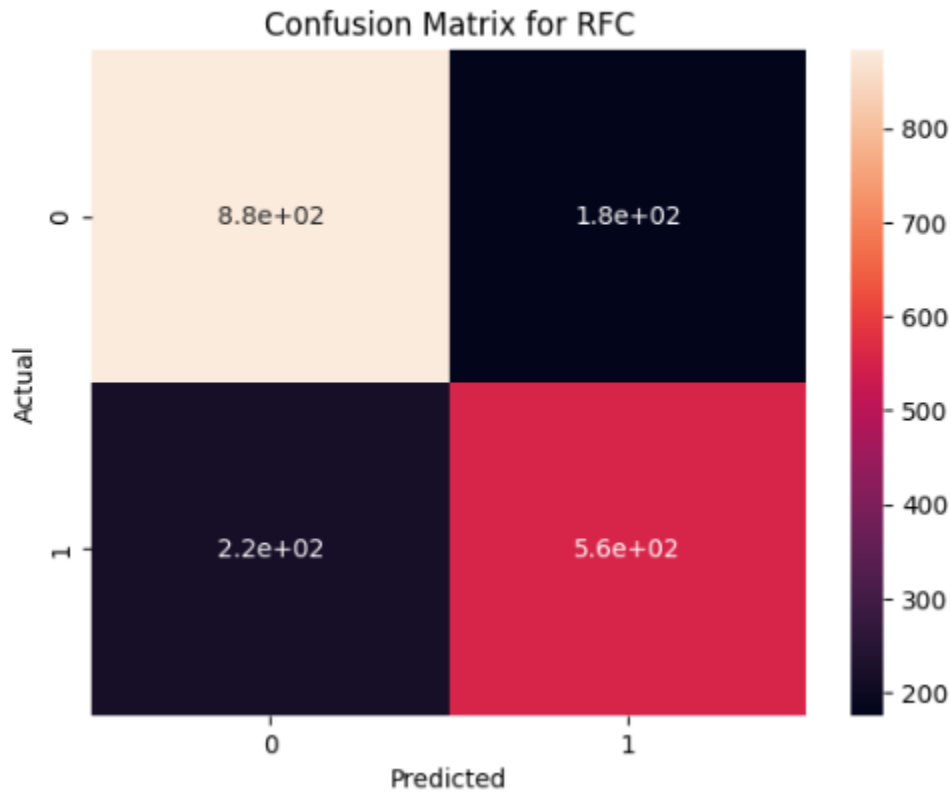
3) **Logistic Regression (Recall: 0.7301)** Logistic Regression had comparable recall to SVM and RF with **FN =216**. Its main strengths:
   a) Simplicity and interpretability.
   b) Efficient training on large datasets.
   c) Although not the best in recall, its consistent performance and ease of integration make it a strong model.

```
[[883 176]
 [216 561]]
```



4) **Random Forest (Recall: 0.7294)** Random Forest slightly slower compared to others but performed well with a recall score and FN **=220**. Despite being a powerful ensemble method, it:
   a) Works slower with high dimensions datasets.
   b) May overfit unless well-regularized.
   c) Still, it can model complex interactions, and with hyperparameter tuning or dense embeddings, performance may improve.

```
[[883 176]
 [220 557]]
```



Model Recommendation for Real World

1. Model Recommended

While Naive Bayes achieves the **highest recall** (0.7568), indicating it misses fewer fake news cases, it suffers from a high number of false positives, which may reduce its reliability in real-world applications.

Logistic Regression and **Support Vector Classifier (SVC)** provide more **balanced performance** across recall, precision, and F1-score, making them better suited for real-world deployment where both **accuracy and efficiency** are critical. They also offer better **scalability** and **interpretability**, especially when integrated into large-scale systems.

Random Forest Classifier, although competitive in terms of FN and FP rates, is **less efficient** on large text datasets due to higher computational demands and therefore is not recommended for deployment in production environments.

- Logistic Regression or SVC for general-purpose, scalable fake news detection systems.
- Naive Bayes only when maximizing recall (minimizing false negatives) is the top priority.
- Avoid Random Forest in high-volume, real-time systems due to computational inefficiency.

2. Improvements/ Alternatives to the approach applied

1. UMAP (Uniform Manifold Approximation and Projection) is a **dimensionality reduction** technique used primarily for:

1) Visualizing high-dimensional data in 2D or 3D: Using UMAP to project the **TF-IDF feature matrix** into 2D space for **exploratory analysis** or to visualize how well a model separates fake vs real news.
2) Discovering clusters or structure in data
3) Preserving both local and global relationships in reduced dimensions

2. Use of Pretrained Transformer Models (BERT)

- **Current Limitation**: TF-IDF treats text as a bag-of-words and loses context, syntax, and word order.
- **Improvement**: Models like **BERT**, **RoBERT** understand sentence structure and semantics and can better capture **sarcasm**, **nuance**, and **complex claims** often present in fake news.

- **Impact**: Likely to outperform classical models in F1-score and real-world generalization, especially on ambiguous or cleverly worded misinformation.

3.Incorporation of Metadata Features

- **Current Limitation:** The model ignores additional metadata like speaker, subject, or context.
- **Improvement:** Use multi-input models or feature engineering to include:
  Speaker credibility history (e.g., speaker's past truthfulness)
  Subject category (e.g., health, politics)
- **Impact:** Can boost accuracy by learning patterns.

4. Ensemble Learning

- **Alternative:** Instead of relying on a single model, ensemble approaches (stacking Logistic Regression + Naive Bayes) can leverage the strengths of different models.
- **Impact:** Increases robustness and reduces risk of underperformance in edge cases.

5. Handling Class Imbalance

- **Observation:** The dataset may be slightly imbalanced (more real news than fake).
- **Improvement:** Use class weighting in models like Logistic Regression and SVC. Apply SMOTE (Synthetic Minority Over-sampling Technique) to balance classes.
- **Impact:** Prevents the model from being biased toward predicting "real" more often.

3. Improvement approach applied

To better understand the structure of our high-dimensional TF-IDF vectors, applied UMAP for dimensionality reduction and plotted the data in 2D space. This helps to visually assess how well the model can separate fake from real news based on the textual features alone."

1. Dimensionality Reduction for Visualization

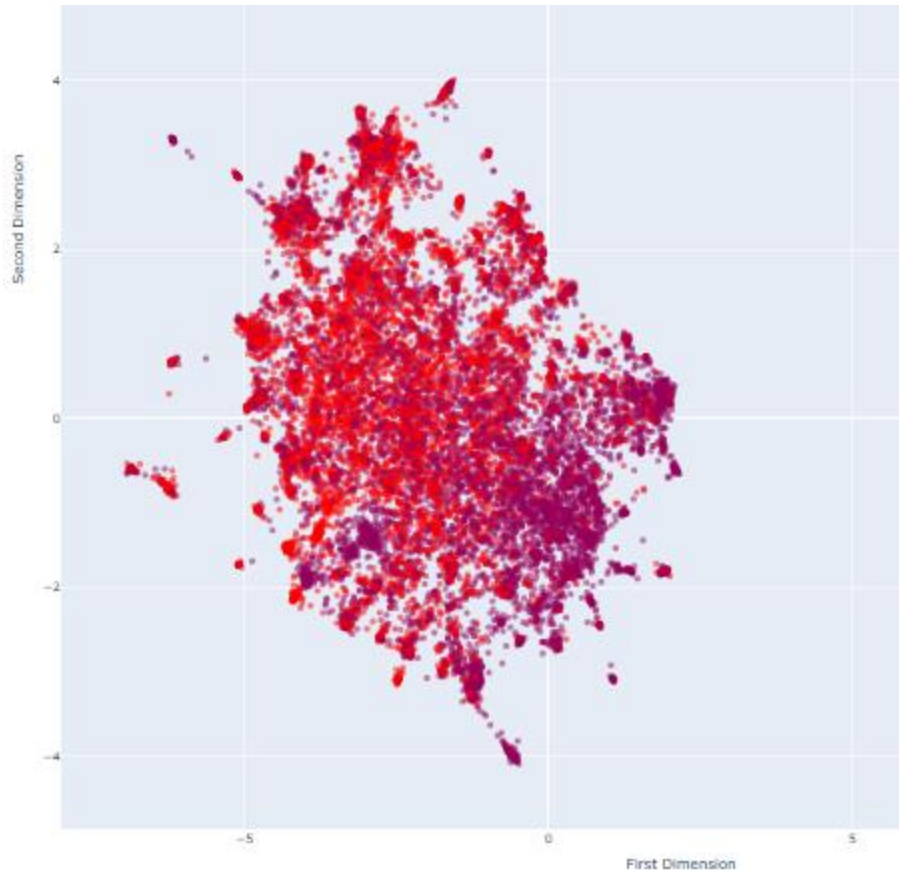
a) The high-dimensional feature vectors were projected into two dimensions (2D) using the following UMAP parameters: n_neighbors=250 and min_dist=0.1. These

parameters were chosen to emphasize global structure while maintaining local neighborhood integrity.

```
# Import necessary visualization and dimensionality reduction libraries
from umap import UMAP
import plotly.figure_factory as ff
import plotly.graph_objs as go
from sklearn.linear_model import LogisticRegression

# Reduce TF-IDF features
# Initialize UMAP with specific parameters
# n_components = 2 → reduce to 2D for visualization
# n_neighbors = 250 → higher value captures more global structure
# min_dist = 0.1 → controls the spacing of points; lower means tighter clusters
umap_model = UMAP(n_components=2, n_neighbors = 100, min_dist = 0.01)
x_umap = umap_model.fit_transform(data_tfidf) # Fit UMAP on the TF-IDF feature matrix and transform it into 2D
statements = data['statement'] # Retrieve original text and labels for hover text in the plot
label = data['label']

# Create a 2D scatter plot using Plotly
# Points are colored using the binary label Y (0 = Fake, 1 = Real)
dataset_umap = [go.Scatter(x =x_umap[:,0],y = x_umap[:,1], mode = 'markers',
                           marker = dict(color = Y, colorscale = 'Rainbow', opacity = 0.5),
                            text = [f'statments: {a}<br>label: {b}' for a, b, in zip(statements,label)],
                            hoverinfo = 'text')]
# Define layout settings for the UMAP plot
layout_umap = go.Layout(title = "UMAP Dimensionality Reduction", width = 1400, height = 1400,
                        xaxis = dict(title = "First Dimension"),
                        yaxis = dict(title = "Second Dimension"))
fig_umap = go.Figure(data = dataset_umap,layout = layout_umap)
fig_umap.show()
```



b) The above scatterplot presents a **2D UMAP projection** of the TF-IDF feature vectors from the LIAR2 dataset. Each point represents a news statement, with colors indicating their binary labels.

c) While there's a high degree of overlap, some **clusters of fake and real news** are distinguishable.

d) This suggests that the textual content of fake news tends to form dense, interpretable regions in the feature space.

e) The clustering pattern supports the notion that **linguistic patterns** in statements carry important signals for fake news detection.

f) UMAP was used strictly as a **visualization tool** to explore the feature space and assess the potential of the input data to support binary classification. It was **not used in training or evaluation** of the classifier itself.

Real-Time Prediction Using Saved Model

Model Deployment Simulation with **joblib**

To demonstrate the applicability of my fake news detection model in real-world scenarios, I used Python's **joblib** library to serialize the trained model. This enabled me to save the model once trained, and then load it for making predictions on new, unseen text samples including randomly generated fake news statements.

This simulates a deployment pipeline, where a model can be loaded in a production environment (e.g., a web app, API, or automation tool) and used for real-time classification of news articles.

News Sample:

1. Government announces plan to abolish elections and appoint leaders directly through AI algorithms starting next year.
2. New study shows that drinking bleach can cure COVID-19 within 24 hours, scientists confirm.

```python
# Save the best performing model (MultinomialNB)
joblib.dump(model, 'multinomial_nb_model.pkl')
joblib.dump(tfidf, 'tfidf_vectorizer.pkl')

print("Model and TF-IDF vectorizer saved.")

# Load the saved model and vectorizer
loaded_model = joblib.load('multinomial_nb_model.pkl')
loaded_tfidf = joblib.load('tfidf_vectorizer.pkl')

# --- Placeholder for predicting on new data ---
# Example of new data (replace with your actual new data)
new_data = ["Government announces plan to abolish elections and appoint leaders directly through AI algorithms starting next year.", "New study shows that drinking bleach can cure

# Preprocess the new data using the loaded TF-IDF vectorizer
new_data_tfidf = loaded_tfidf.transform(new_data)

# Make predictions using the loaded model
predictions = loaded_model.predict(new_data_tfidf)

print("\nPredictions on new data:")
for statement, prediction in zip(new_data, predictions):
  label = "Real News" if prediction == 1 else "Fake News"
  print(f"Statement: '{statement}' -> Predicted Label: {label}")
```

Output:

```
Model and TF-IDF vectorizer saved.

Predictions on new data:
Statement: 'Government announces plan to abolish elections and appoint leaders directly through AI algorithms starting next year.' -> Predicted Label: Fake News
Statement: 'New study shows that drinking bleach can cure COVID-19 within 24 hours, scientists confirm.' -> Predicted Label: Fake News
```